Motivation & Goals

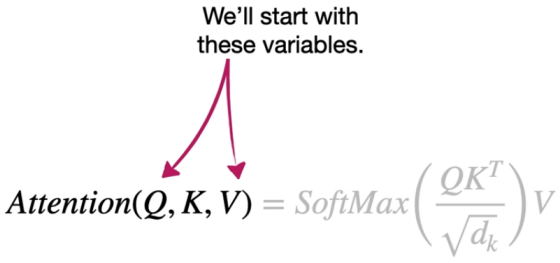
- Why transformers? Limits of RNNs/LSTMs/GRUs for sequence tasks;
    - parrallel verses sequential processing
    - self-attention - they handle long range dependencies better
- RNNs Forget information in long sequences
- RNNs have a vanishing gradient problem
    - Activation function like tanh (-1<g<1) typically less than 1, say its .1
    - if 10 cycles thru; (.1)**10=0.0000000001

- Learning objectives for the session.
    - Tokenizer (BPE example) tiktoken.com?
    - Embeddings
    - Attention
    - Coding Attention

Tokens, Embeddings, Positions

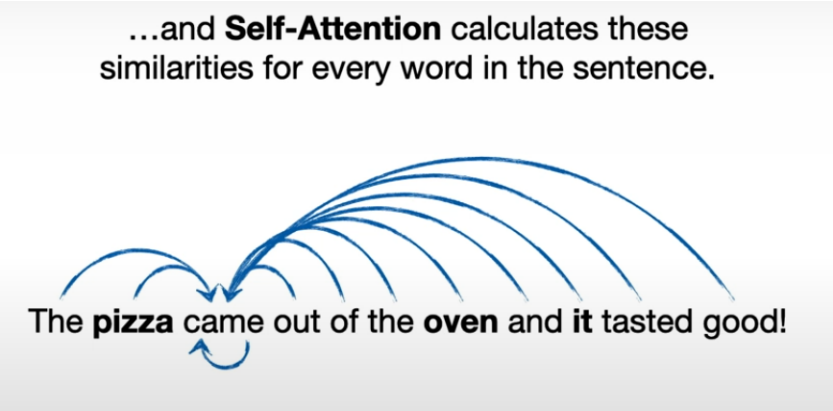
- Tokenization (BPE), vocabulary, special tokens.
- See BPE_example.pdf
- Embedding matrices; positional encodings (sinusoidal skip for this).
    1) Just a table lookup, they are trainable
- Quick “shape math” (batch, seq_len, d_model).

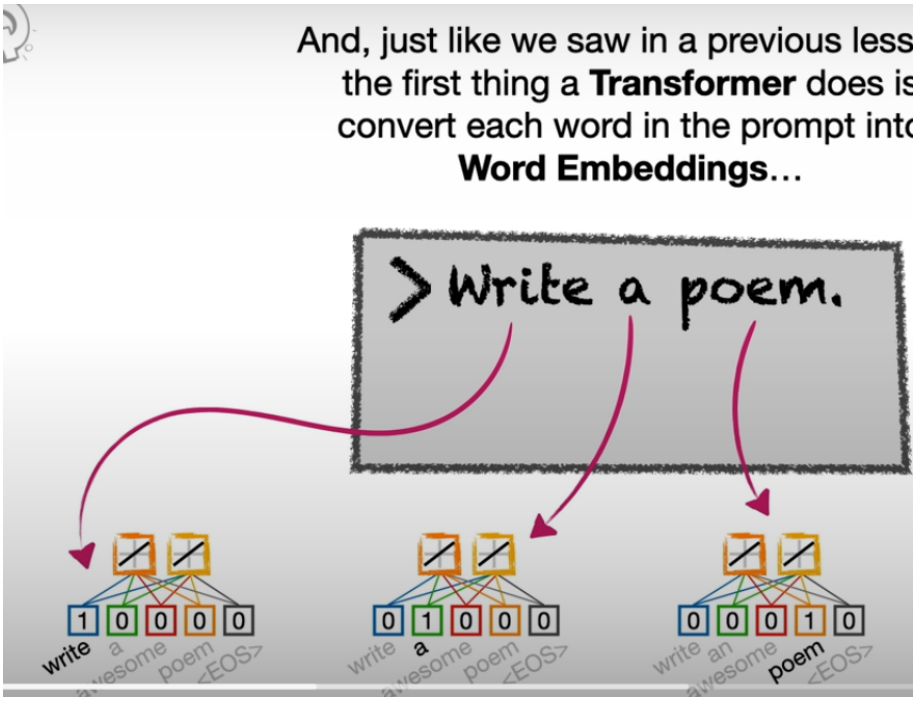
Self-Attention Core

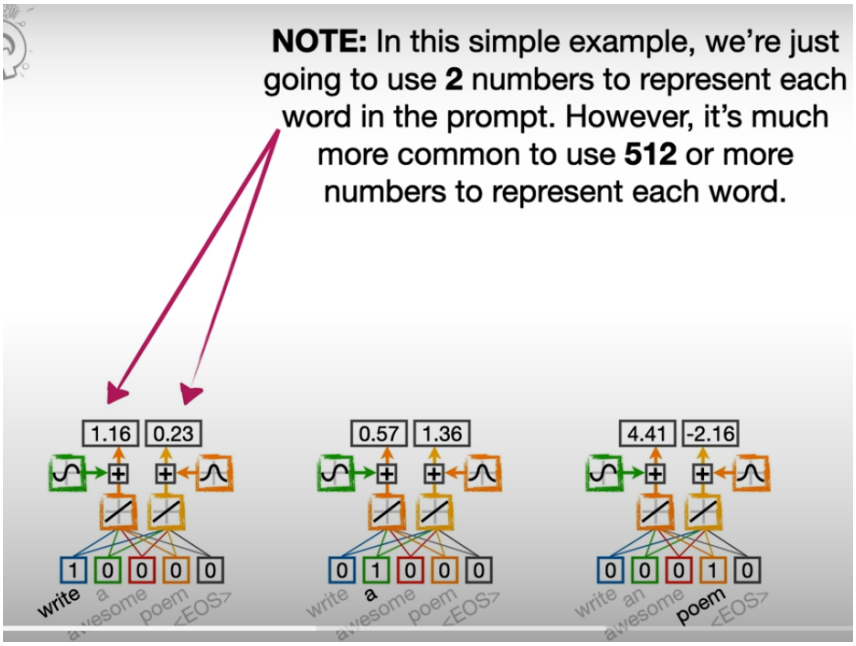


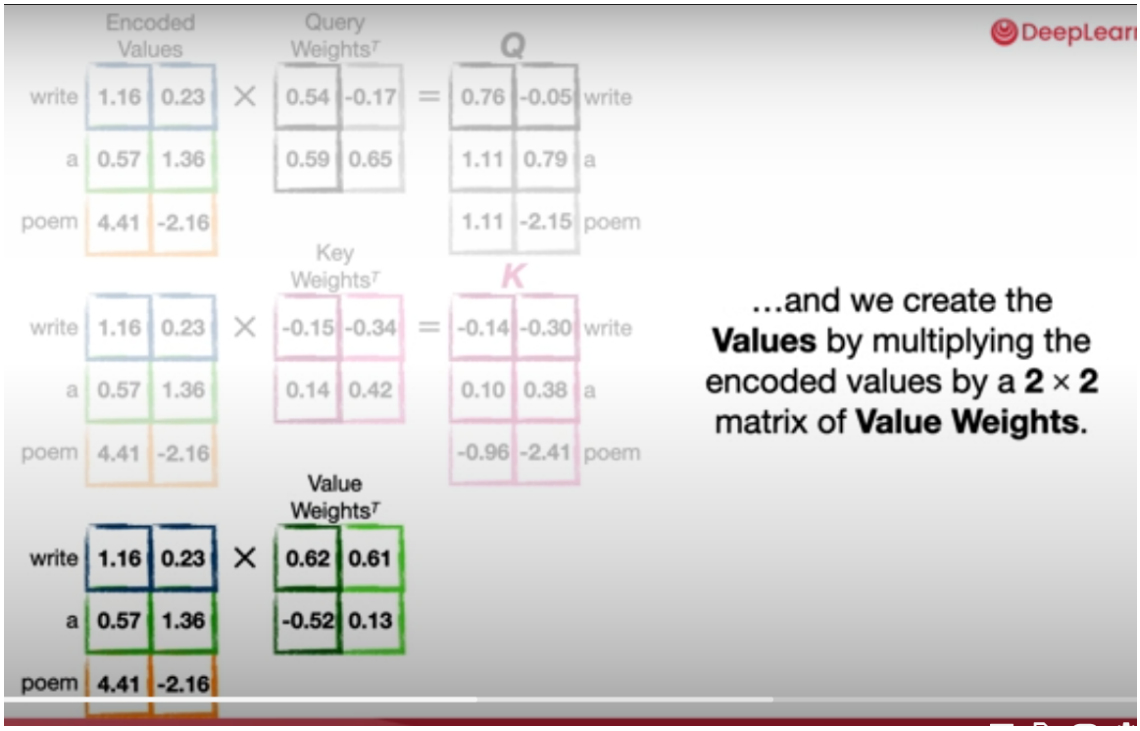
- Query- what I am looking for
- Key – what do I contain
- Q@Kt is a dot product, will give the similarity between Q and K (or how much attention should I pay)

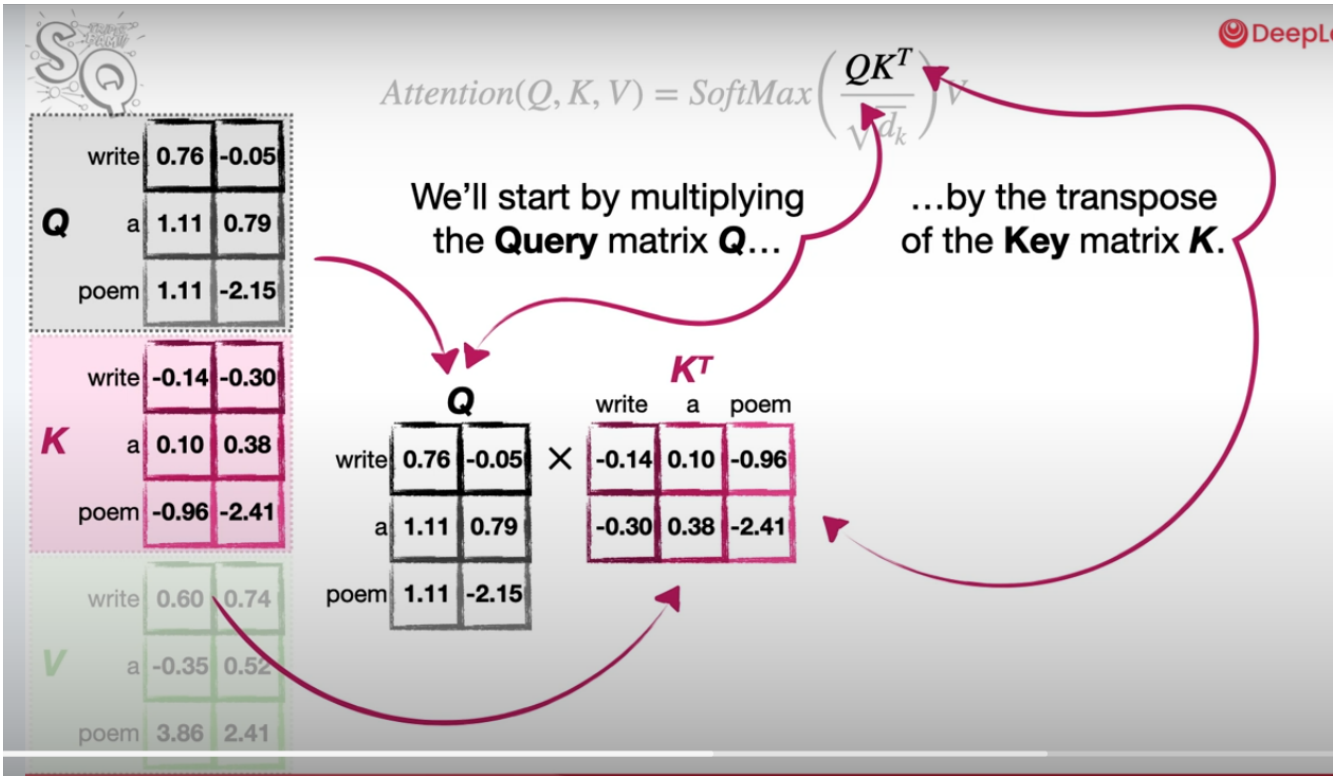
- The dk**-2 makes sure the values are small so the softmax results in diffuse probabilities (if large softmax would degenerate into 1 hot encodings, so no attention to any but one)
- Then @V will give the weighted sum of all values



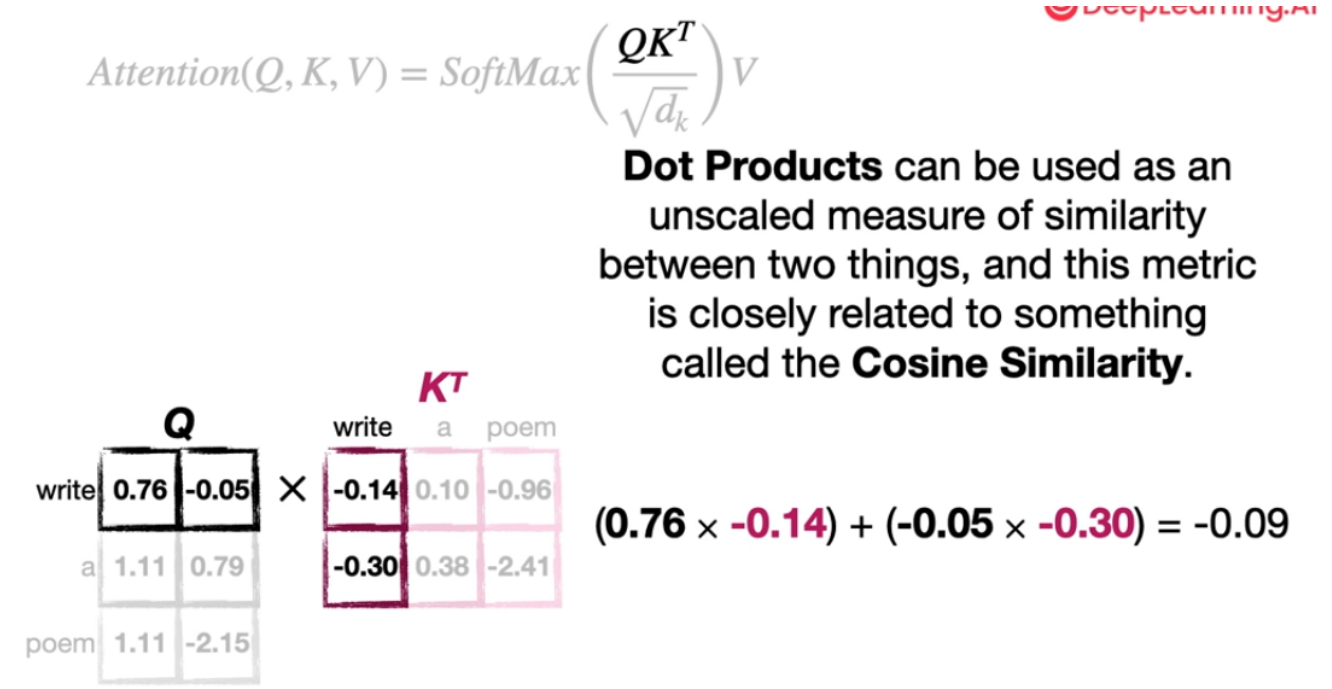



NOTE: In this simple example, we're just going to use 2 numbers to represent each word in the prompt. However, it's much more common to use 512 or more numbers to represent each word.
1.16
0.23
0.57
1.36
4.41
-2.16
write
a
awesome
poem
an
<EOS>

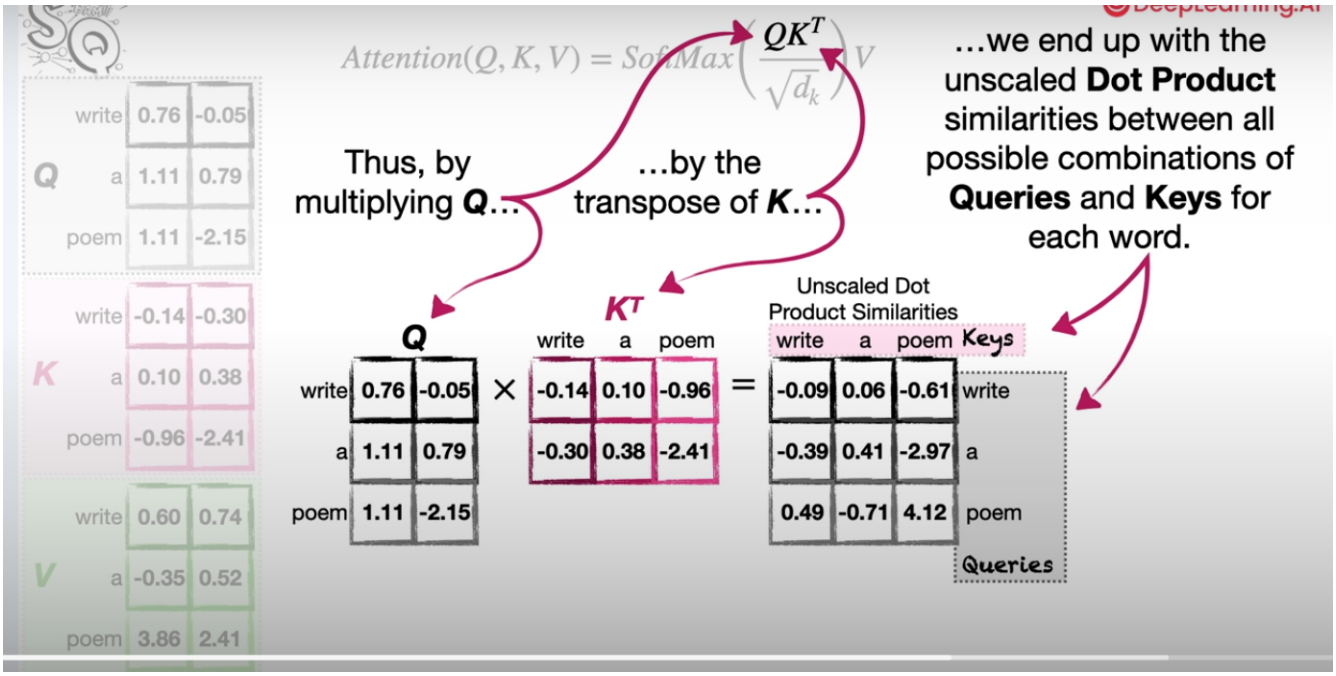

Encoded Values
Query Weights^T
Q
write 1.16 0.23 × 0.54 -0.17 = 0.76 -0.05 write
a 0.57 1.36 0.59 0.65 1.11 0.79 a
poem 4.41 -2.16 1.11 -2.15 poem
Key Weights^T
K
write 1.16 0.23 × -0.15 -0.34 = -0.14 -0.30 write
a 0.57 1.36 0.14 0.42 0.10 0.38 a
poem 4.41 -2.16 -0.96 -2.41 poem
Value Weights^T
write 1.16 0.23 × 0.62 0.61
a 0.57 1.36 -0.52 0.13
poem 4.41 -2.16
...and we create the Values by multiplying the encoded values by a 2 × 2 matrix of Value Weights.



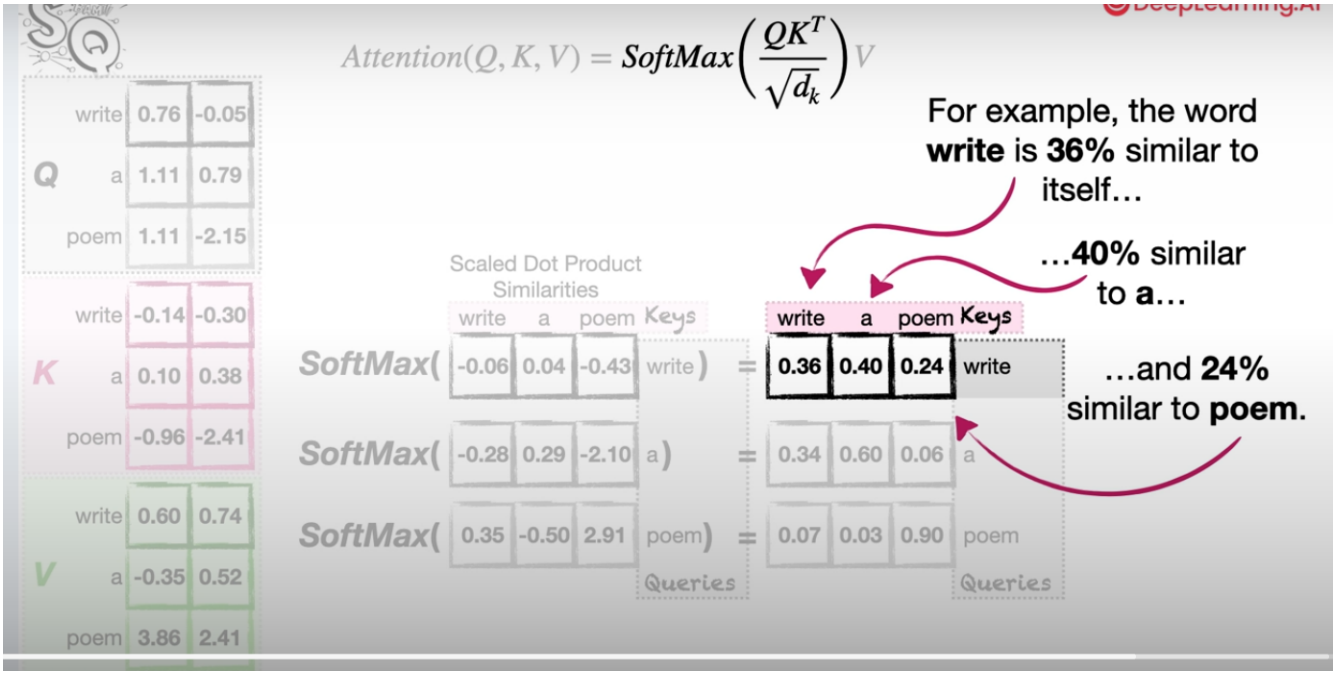
The Q@K is a dot product similarity score (show vectors, and what cosign similarity means)

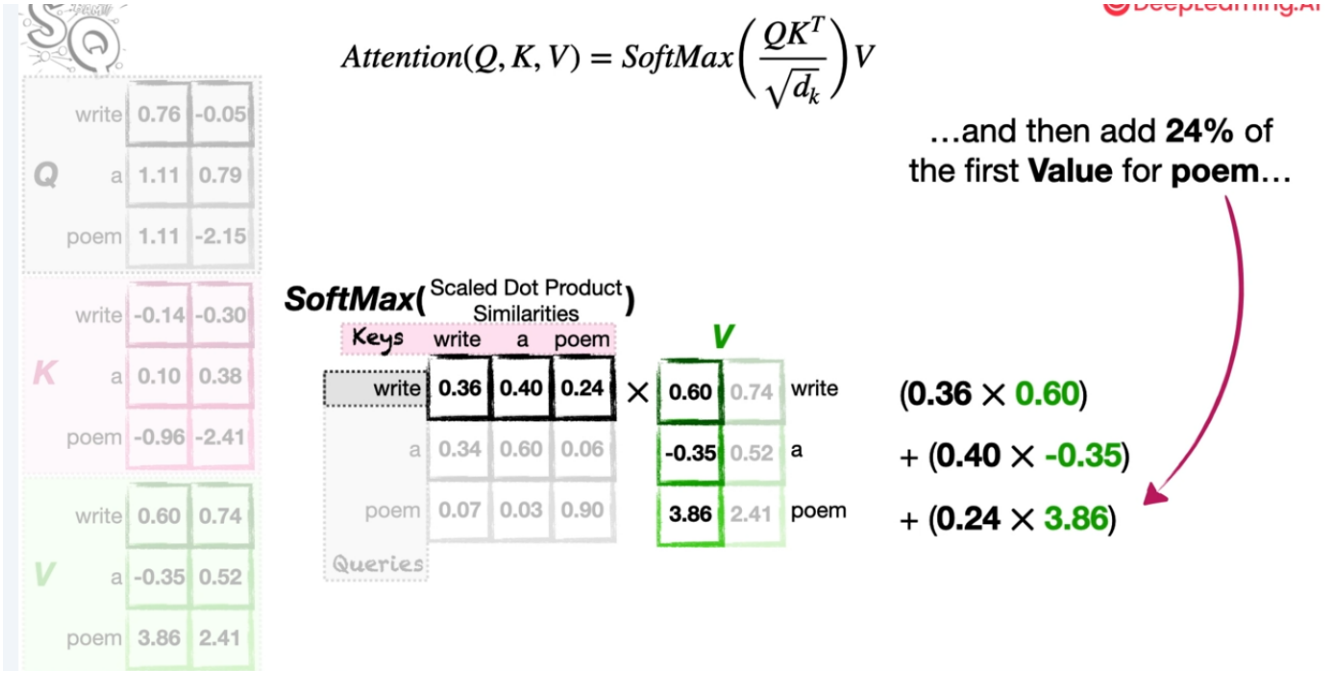


but the dot product isnt scaled (hence the dk-2 term to make sure values are diffuse)**



dk is dimension of key matrix (length of embedding)





The softmax percentages multiplied by V matrix is a weighted sum of all words, ones that are most similar have more of their value added in

Attention

- How to represent this in Pytorch?
- A Linear Layer! (See notebook)

- How to code this equation?(See notebook)

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$